T4D AI COHORT 2

Building Generative AI Workflows:

Our journey building Avni AI App configurator

The Problem

Why a chat-first AI configurator for Avni

Manual webapp config

Click-by-click through Avni UI.
Avni expertise required. Consistency on the user.

IDE-based AI (Cursor, Claude Code)

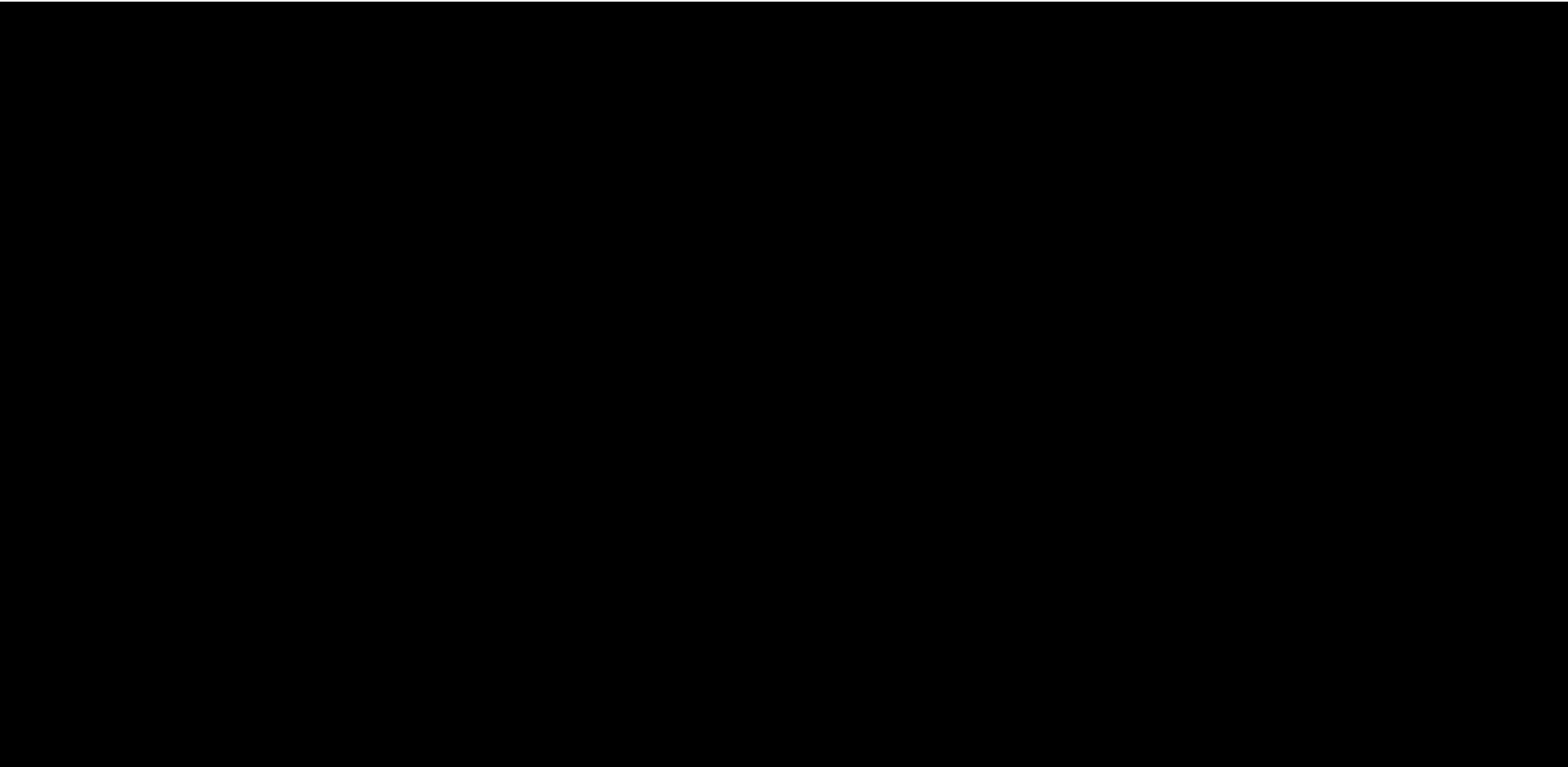
Developer-only. Decoupled from Avni.
Quality varies with AI + user effort.

Our AI App Configurator — chat-first, not click-first

- Chat-first. No webapp, no code, no Avni expert.
- Program managers drive the setup themselves..
- Schema validation + guardrails in the flow, not LLM judgment.
- Deterministic pipeline → reproducible, auditable bundles.
- Earlier it used to take few hours for average form creation, now that gets done in few minutes
- Usage is high during initial setup and sporadic usage later on

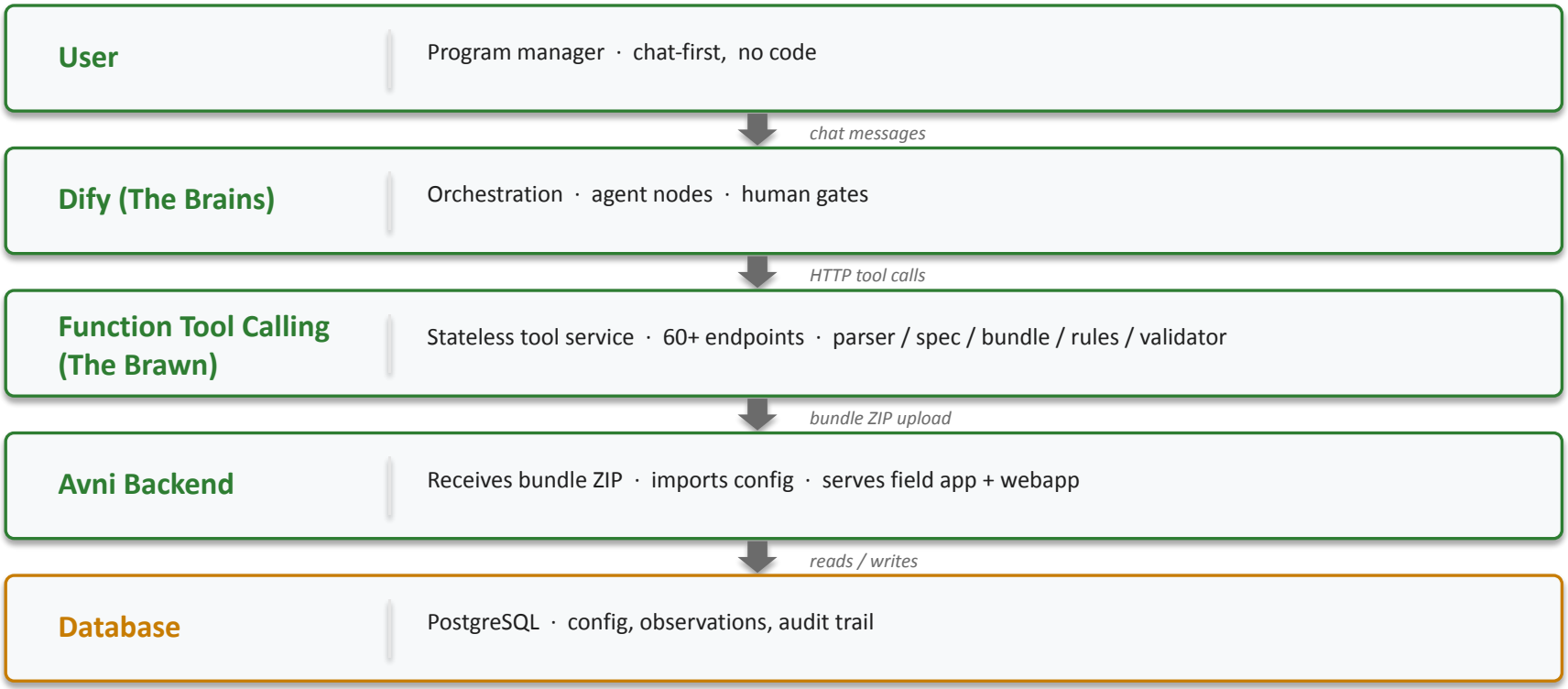
Looks like an LLM problem. Most of it is deterministic.

Avni App Configuration complexity



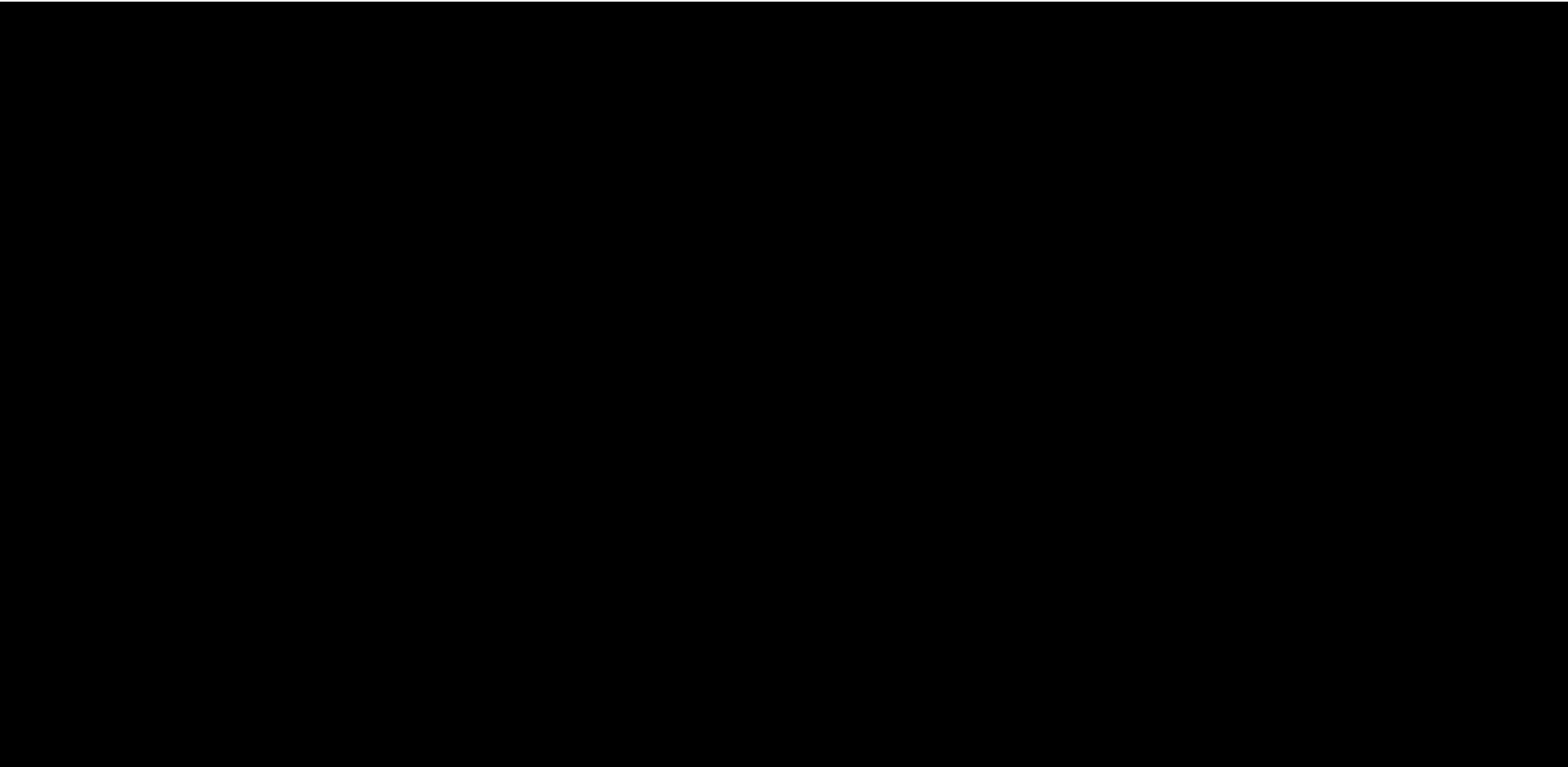
What We Built

Tier architecture — user chat down to the database



Deterministic on the rails, agentic in the gaps.

Avni App Configurator Demo



Lesson 1 — WHERE to put the LLM

System-level architecture · deterministic code + LLM agents + variables

Write the contract first. Writable → deterministic. Fuzzy → scoped agent.

Go deterministic when

- Schema is writable
- Reproducibility matters
- Validation expressible using logic
- LLM cost not justified

e.g. parsing · validation · state management

Use an LLM agent when

- Input shape varies
- Judgment or reasoning needed
- User dialog / clarification
- Open-ended synthesis or correction

e.g. classification · dialog · generation · auditing

THE GLUE — variables thread state between the two sides

session ids · phase flags · approvals · retry counters · pause signals

Can write the contract → deterministic. Can't → agent.

Avni Spec Configuration Template

```yaml
# Registration form
registrationForm:
  decisionRule: "..."          # Form-level decision rule
  visitScheduleRule: "..."     # Form-level visit scheduling
  validationRule: "..."        # Form-level validation
  checklistsRule: "..."        # Form-level checklist rule
  sections:
    - name: "Basic Details"
      fields:
        - name: "Field Name"
          dataType: Text  # Text|Numeric|Coded|Date|DateTime|Duration|PhoneNumber|Audio|Image|ImageV2|Video|File|Location|Subject|GroupAffiliation|Id|Time|Notes|QuestionGroup
          selectionType: Single  # Single | Multi (for Coded fields)
          mandatory: true
          # Numeric-specific
          min: 0                       # lowAbsolute
          max: 100                     # hiAbsolute
          lowNormal: 10                # Normal range lower bound
          hiNormal: 90                 # Normal range upper bound
          unit: "kg"
          # Coded-specific
          options: ["Option A", "Option B"]
          # Skip logic
          skipLogic:
            dependsOn: "Other Field"
            condition: "="
            value: "Yes"
          # Key-value metadata
          keyValues:
            editable: false
            ExcludedAnswers: ["uuid1"]
          # Validation format
          validFormat:
            regex: "^[6-9]\\d{9}$"
            descriptionKey: "Must be 10 digit mobile number"
          # Element-level rule (JavaScript)
```

Lesson 2 — HOW to shape the agents

Agent-level design · sub-agents, divide and conquer

One big agent → specialist sub-agents. Smaller brain. Clearer job. Surgical edits.

Small context

Each agent sees only its slice.

Low tokens

Small contexts × many agents
= cheap. Caching wins.

One specialist, one job

Clear I/O contract.
Bounded iterations.

Surgical edits

Edit one file, one rule.
Never regenerate the whole.

Common sub-agent archetypes

Classifier

Route input to
the right path.

Planner

Break task into
ordered steps.

Synthesizer

Produce the
artifact as per the spec.

Patcher

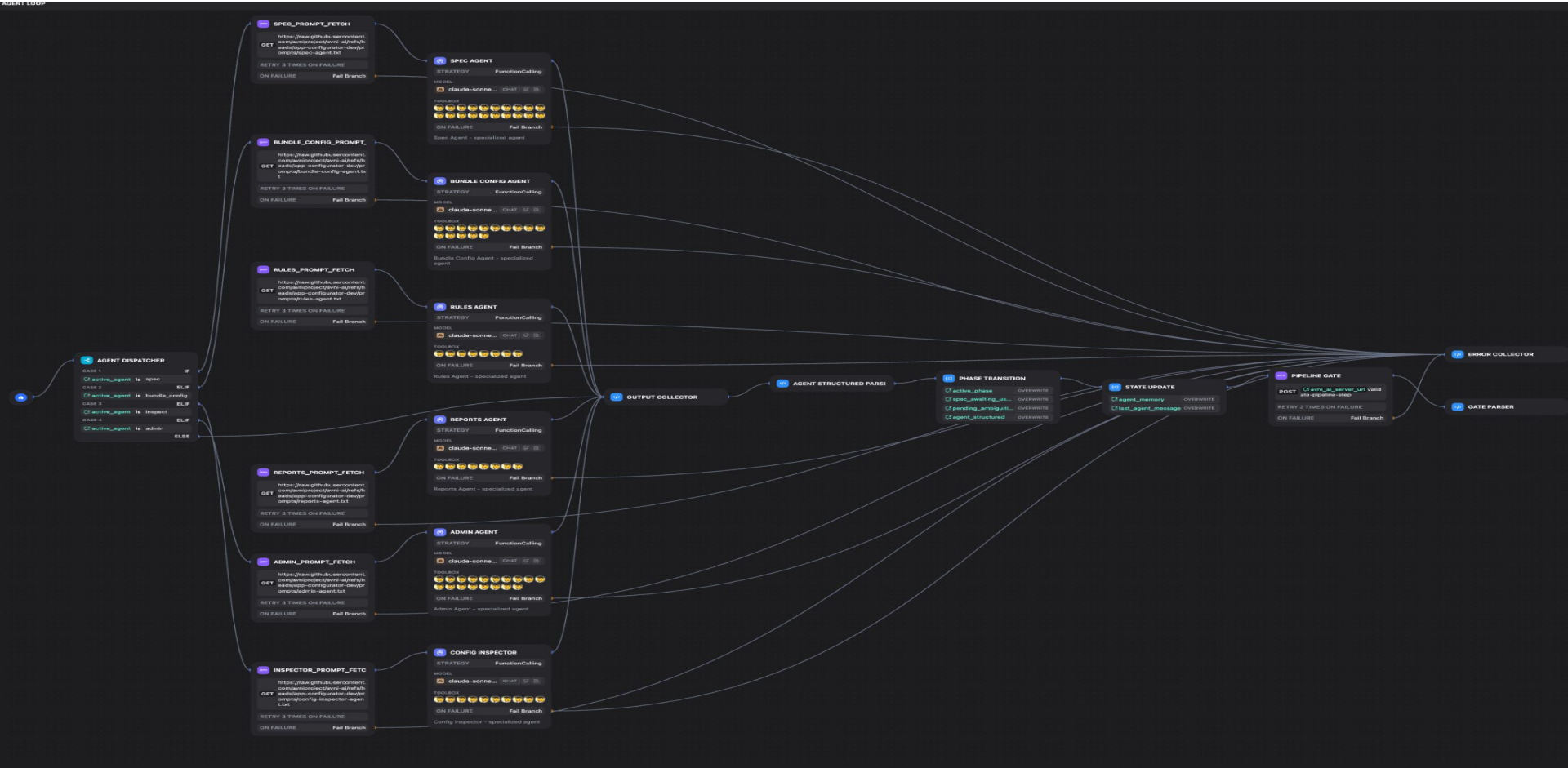
Edit one part.
Not regenerate.

Inspector

Audit output.
Read-only critic.

Small brain. Clear job. Surgical edits.

Avni App Configurator Agentic Workflow



Lesson 3 — HOW to design the tools

Tool-level design · keeps the LLM cheap, fast, capable of depth

Design every tool assuming an LLM will call it. Return only what's needed.

Sectional retrieval

Return only what this
turn needs. Not the whole base.

Response truncation

Cap outputs.
Send 'top 20 + count'.

Surgical edits

Target + patch.
Not full rewrite.

Stateless

Individual tools do not maintain state within themselves.
But you could have tools to save, modify or retrieve overall state(data).

Handles, not payloads

Heavy state in a store.
LLM sees a handle + summary.

Defensive guards

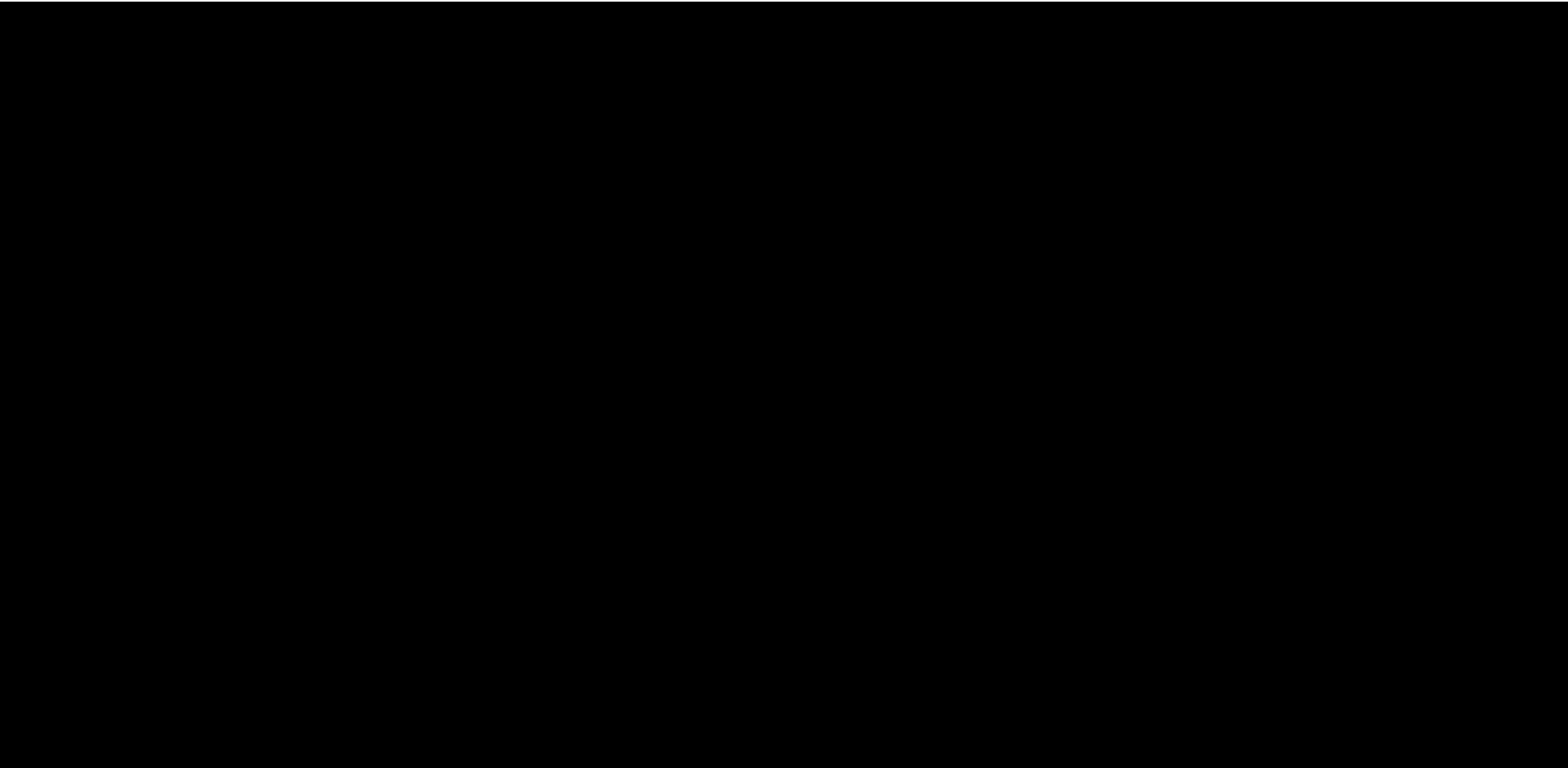
Validate in, validate out.
Bound depth. Fail clean.

Before — LLM exhausted on initial entities. Couldn't go deeper.

With these — 10× fewer tokens per turn. Depth becomes routine.

Design the tool for the Agent that will call it.

Avni AI OpenAPI



What's Next, and What's in the Way

Roadmap · Dify workflow shortcomings

Orchestrating open-ended long-horizon tasks using LangChain/LangGraph/Dify was a lost cause for us. Moving to Claude-managed agents instead.

Shipping next

- Complete the bundle — enhance support for complex configuration
- Handle multi-turn conversation for configuration refinement
- Production org safety — trial-only today
- Adopt cohort guardrails — regex + small-LLM guards
- Benchmark setup time vs. manual webapp config
- Unstructured scoping docs (PDF / image) ⟶ spec

Dify workflow — shortcomings

- Doc extractor misses content — parsing moved to our own parser on avni-ai
- Agent nodes can't call auth'd tools using perishable tokens — token lives in conversation-scoped in-memory store
- No sub-workflow imports — HTTP-self (200–500 ms) or copy-paste across flows
- Prompt length UI limits — inject long prompts as variables
- YAML / visual-flow config — adoption curve for developers used to code files

The Devil is in the details.!

THE NEXT CHAPTER · Take 2 - Side A

avni-skills-sdk

Claude-managed agents

“Claude Code, but for an Avni bundle generation”

The New Architecture — two stages, one workflow

System-level · deterministic generator + Claude Agent SDK refinement loop

Forms.xlsx + Modelling.xlsx in. Ready-for-AVNI bundle out. The LLM only refines.

Stage 1 · Deterministic

- No LLM · < 1 s · free
- 2 Excel files ⟶ 30+ JSON + zip
- Reproducible, auditable bundles
- Runs the validator, not a model

e.g. parse · generate · validate

Stage 2 · Chat-driven refinement

- Claude Agent SDK loop
- Each turn becomes a git commit
- Validator runs after every turn
- Correct, extend, ship — in chat

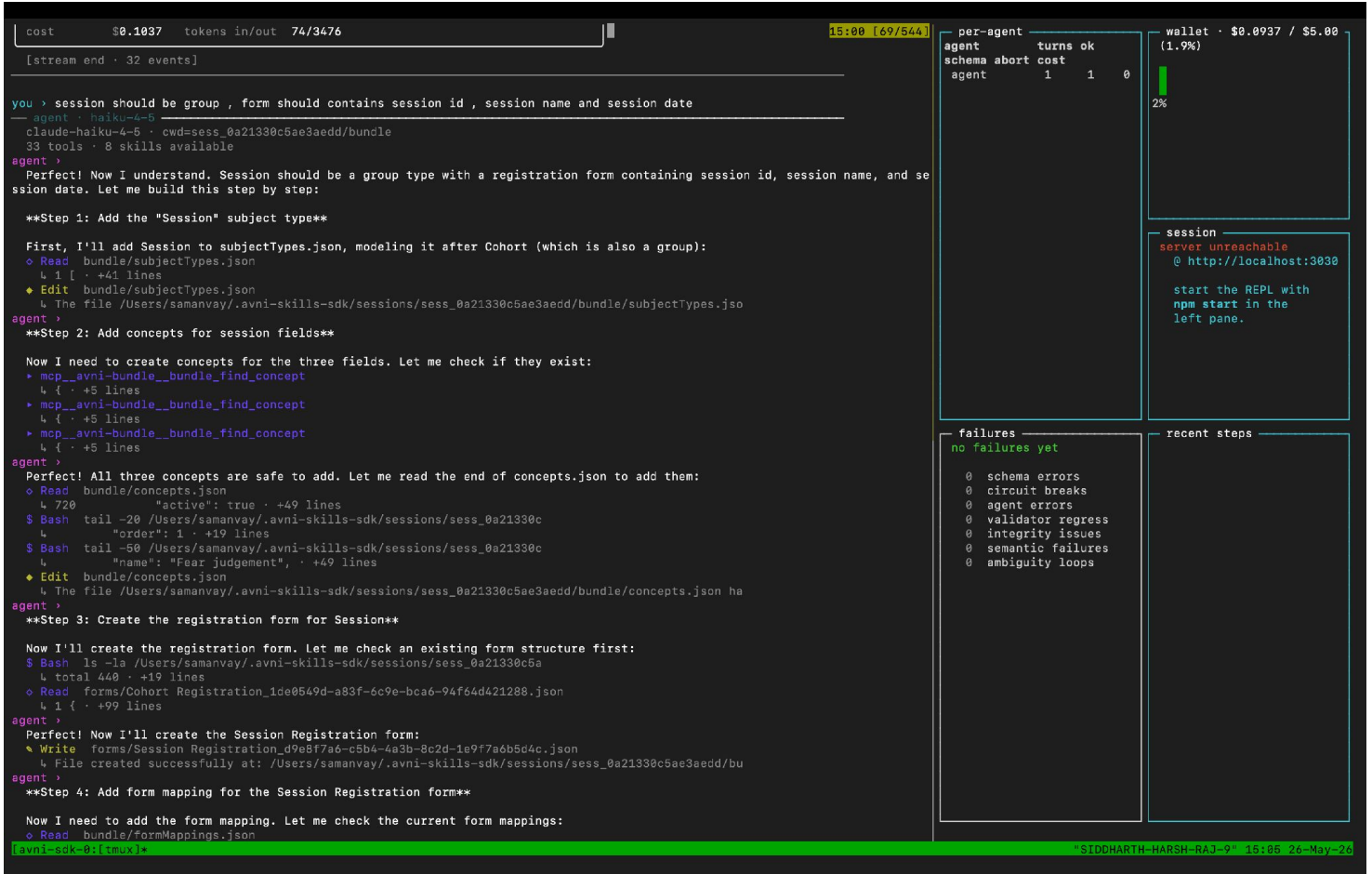
e.g. fix C5 · add subject · rename

THE BRIDGE — slide 12's "moving to Claude-managed agents," realized

deterministic rails stay free & fast · agentic judgement now audited, capped, reversible

Same thesis as before — now without Dify.

Multi-step build — tools run before every edit



Building block 1 — spec in, bundle out, round-trip

Stage 1 · deterministic · avni-skills

One human-readable YAML spec compiles to a full Avni bundle — and decompiles back. Edits patch the live bundle; they never regenerate it.

Spec → bundle

YAML parses to entities;
generator emits 30+ JSON files.

Bundle → spec

Emitter reverses it; parse→emit
is byte-equivalent.

Patch, don't regen

New spec merges onto the live
bundle — UUIDs + UI edits kept.

Deterministic IDs

Hash-seeded UUIDs. Same spec
→ same bundle, every run.

Canonical ZIP order

Files in server load order.
No silent import failures.

21-org schema

Reverse-engineered from 21
prod orgs. Zero missed keys.

Before — bundles hand-built in the UI or regenerated from scratch; edits and UUIDs lost.

Now — a versioned spec compiles, round-trips, and patches in place.

The bundle is a build artifact of a spec — not a hand-edited blob.

Building block 2 — bundle must adhere to a graph

Stage 1 · deterministic · avni-skills

Model the bundle as a UUID graph of foreign keys. Then you can ask what depends on what — and catch broken references before the server ever sees them.

Every entity a node

Concepts, forms, programs, mappings — keyed by UUID.

Every link an edge

Typed references: form element → concept, mapping → program.

Impact check

What breaks if I delete this? See it first.

Prerequisites

What must already exist for this to be valid?

Broken-link scan

Flags every reference to a missing UUID.

Form-type aware

Knows a program encounter needs a program + type.

Before — dangling references slipped through and failed silently on upload.

Now — integrity is checked locally; delete-impact is visible before you act.

Know what breaks before it breaks.

Building block 3 — knowledge the agent can actually use

Stage 1 · Knowledge corpus · avni-readme · avni-ai · avni-skills

Three sources — implementer docs, a merged corpus, and patterns mined from real bundles — compressed into chunked, tagged knowledge the agent pulls on demand.

Docs as source

203 implementer guides (~21k lines) from avni-readme.

Merged corpus

Assembled into merged.md — ~557 KB, ~22.5k lines.

Compressed + chunked

Split into 68 topic files with semantic chunks.

Metadata-tagged

Keywords, audience, retrieval-boost per chunk.

Patterns from prod

4,949 concepts + 132 rule templates mined from real orgs.

Retrieval-ready

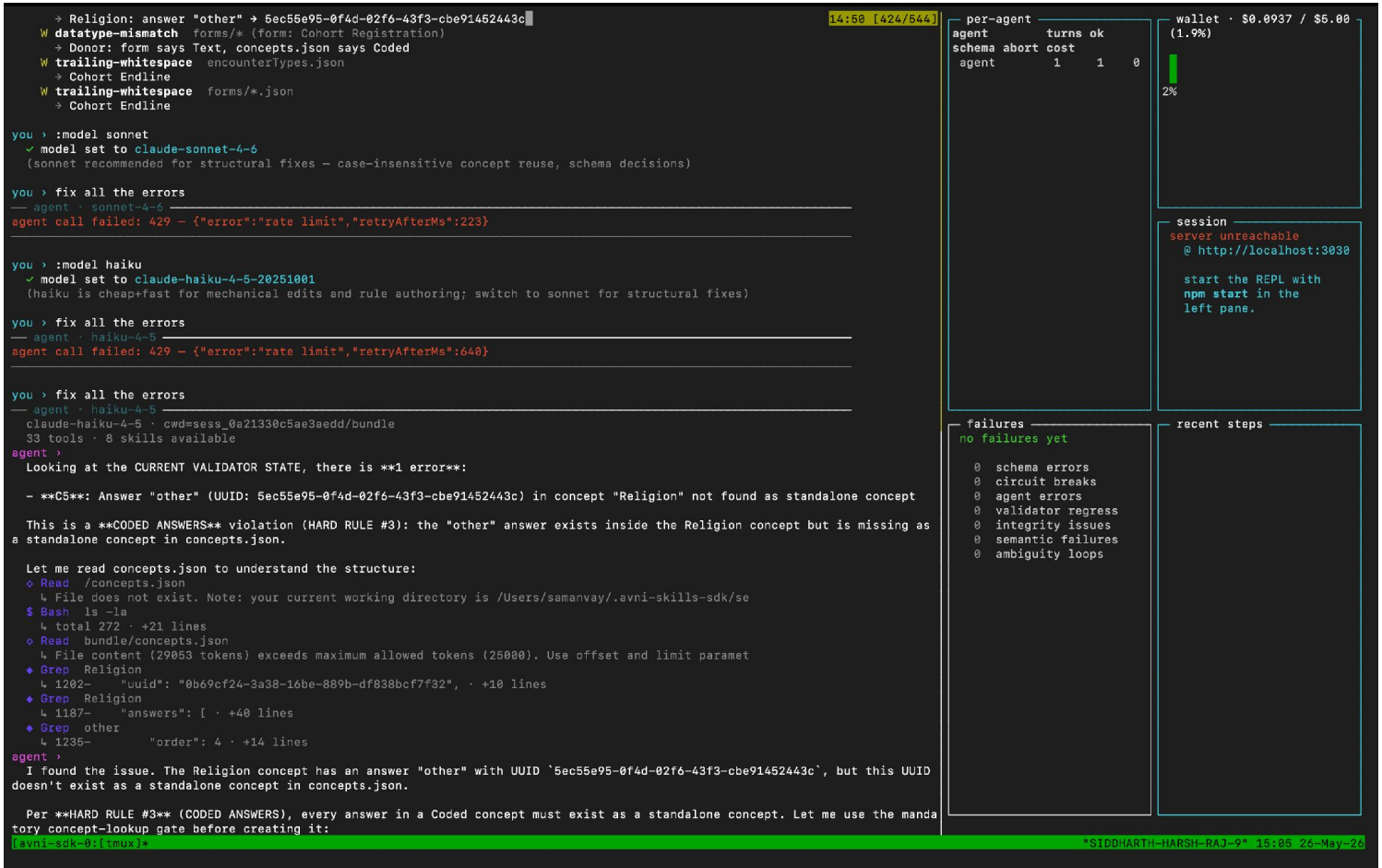
Agent pulls the right topic, not the whole monolith.

Before — one 557 KB monolith; retrieval dragged in noise — ~half was infra/reporting.

Now — 68 focused, tagged topics (~96% implementer) + mined patterns.

Compress for retrieval — high signal beats raw volume.

Validator-state injection + :model switching mid-session



Building block 4 — logged, gated, capped

Stage 2 · agentic SDK · avni-skills-sdk

Assume the agent will try everything. Make every turn recorded, every destructive path blocked, and every dollar capped — by construction, not by trust.

Git-backed history

One commit per turn — a git repo.
Diff, revert, or resume any turn.

Audit by construction

Validator state in every prompt;
self-corrects on resume; durable.

Destructive ops gated

Hook blocks git / rm -rf / sudo;
a detector reverts foreign commits.

Cost capped

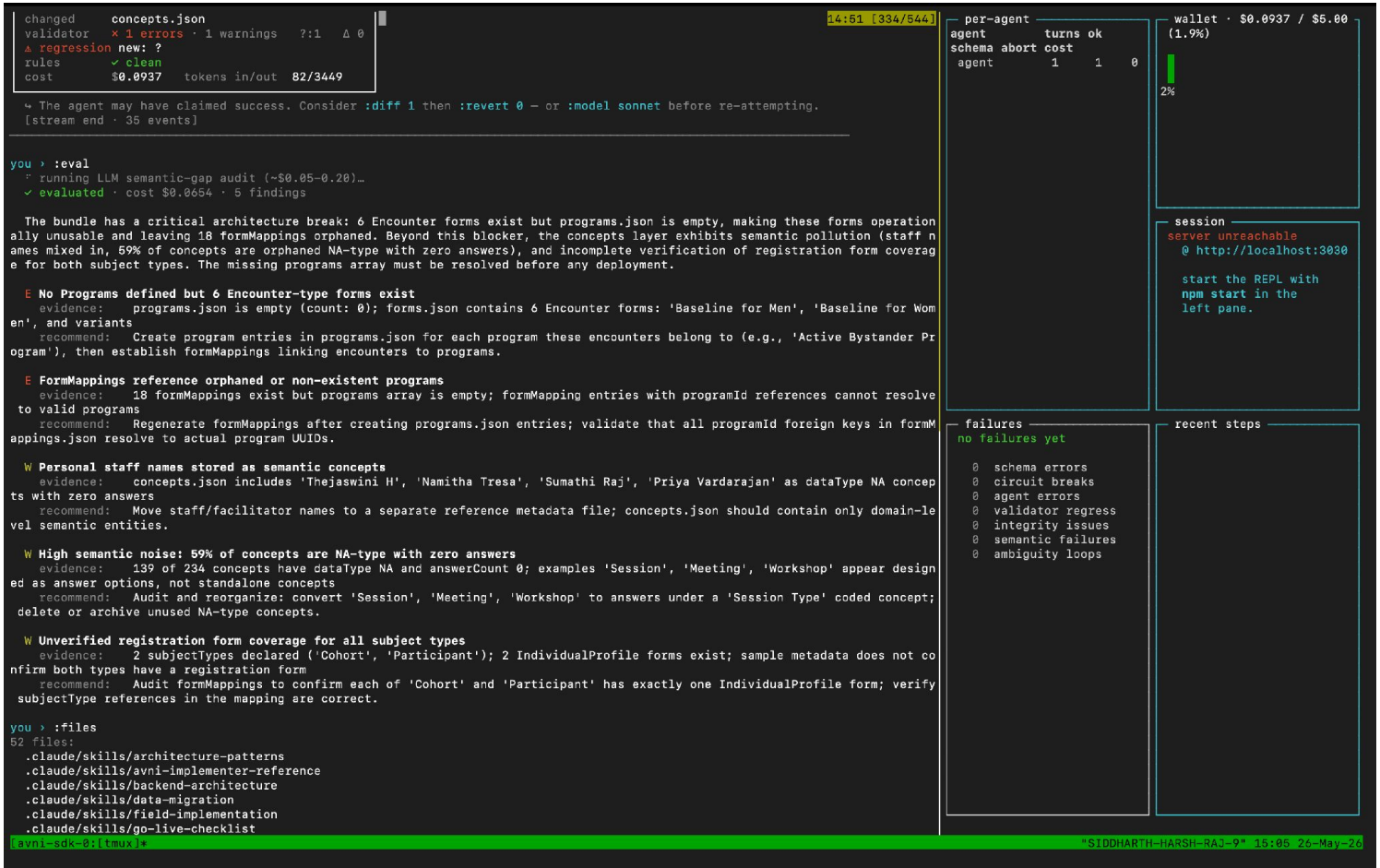
Hard wallet: $5/session, $1/turn;
aborts a no-edit token spree.

Before — state lived in chat memory; an agent could wander or burn the budget.

Now — every turn is git-backed and auditable; destructive paths gated; spend capped.

Audit-rigorous and bounded — by construction.

Regression detection + :eval LLM semantic audit



Building block 5 — prefer the tools, observe everything

Stage 2 · agentic SDK · avni-skills-sdk

Give the agent purpose-built tools it reaches for instead of raw shell — and make everything it does observable, three ways.

Prefer tools over shell

Four in-process bundle tools:
validate, find-concept, summary, export.

Determinism on rails

Deterministic :summary handles routine;
LLM :eval audits the gaps.

Captured three ways

On-disk JSONL, live HTTP
endpoints, and REPL commands.

See it live

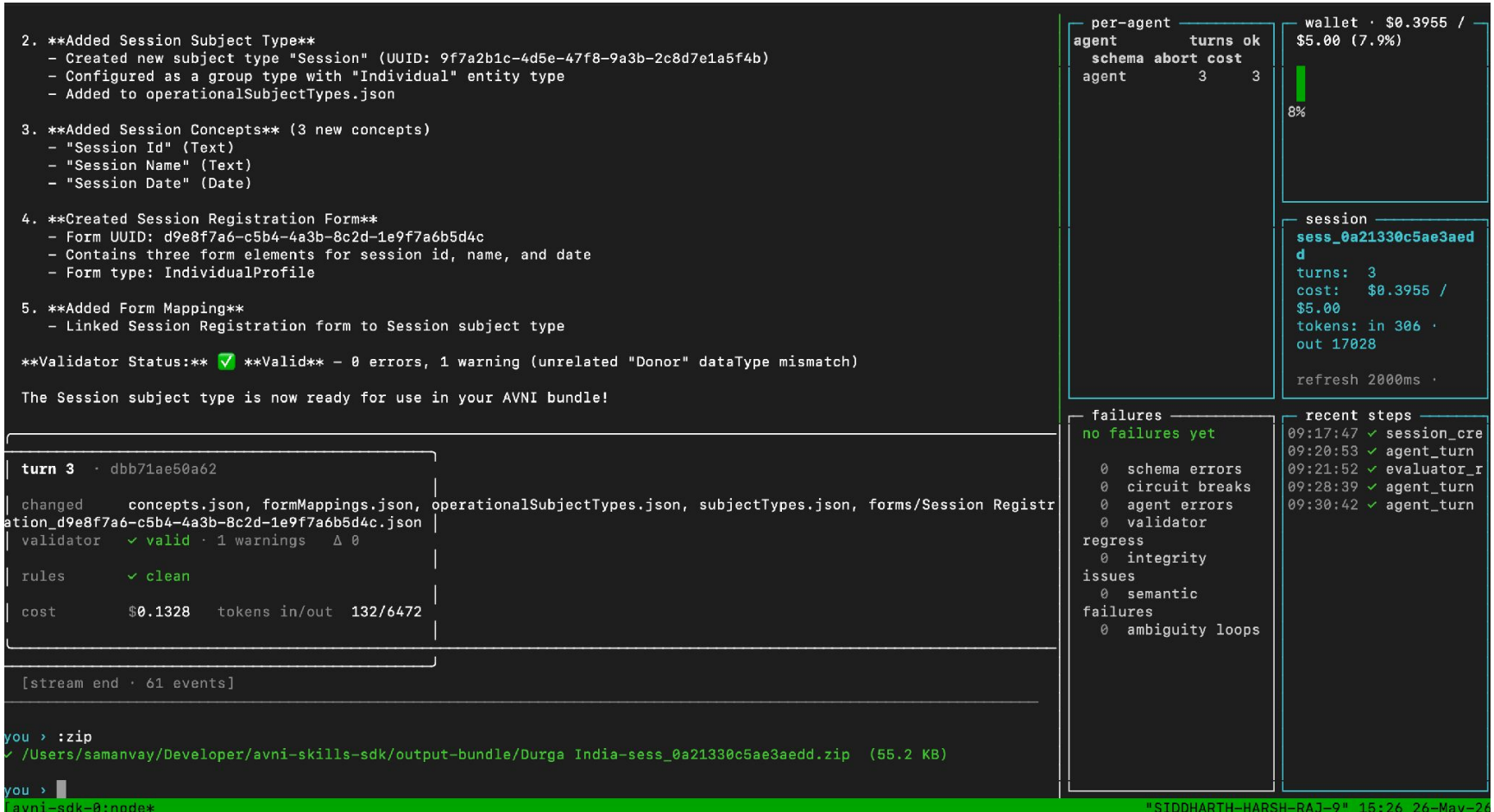
A live dashboard: per-agent
cost and failures by category.

Before — the agent improvised with shell; runs were opaque and hard to debug.

Now — it prefers a few bundle tools, and every action is logged, served, inspectable.

Deterministic checks on rails; LLM judgement only in the gaps.

One bundle, end to end · :zip export · $0.13



THE NEXT CHAPTER · Take 2 - Side B

avni-autopilot

Agentic workflows with Langraph

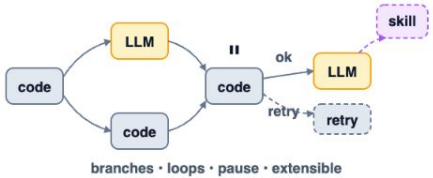
Orchestration framework choice

Three approaches, one shape that fits our pipeline.

OUR PICK

LangGraph

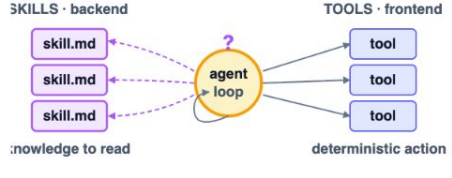
CODE-FIRST STATE MACHINE



- Mix **deterministic code + LLM** in one graph, branch + loop as needed
- **Skills can plug in** as nodes when useful — not forced, not locked out

Claude Agent SDK

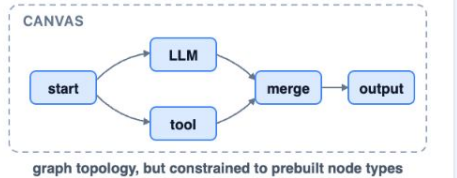
LLM-DRIVEN AGENT LOOP



- **Skills** are reusable knowledge docs (backend); **tools** are the user-facing actions (frontend)
- Using skills for **open-ended editing / fixing** is **non-deterministic** — the LLM interprets the skill on the fly, same prompt can pick different actions
- **Limited observability** — hard to tell which skill was picked or used

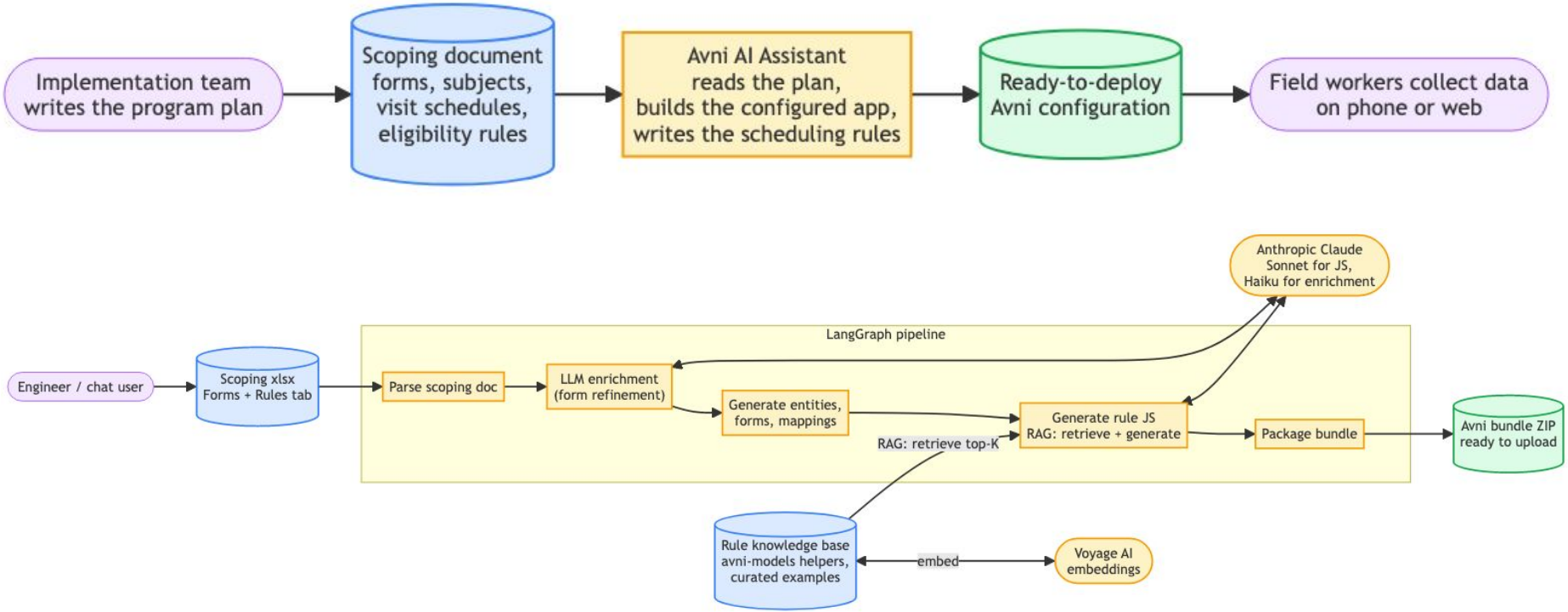
Dify

VISUAL LOW-CODE CANVAS

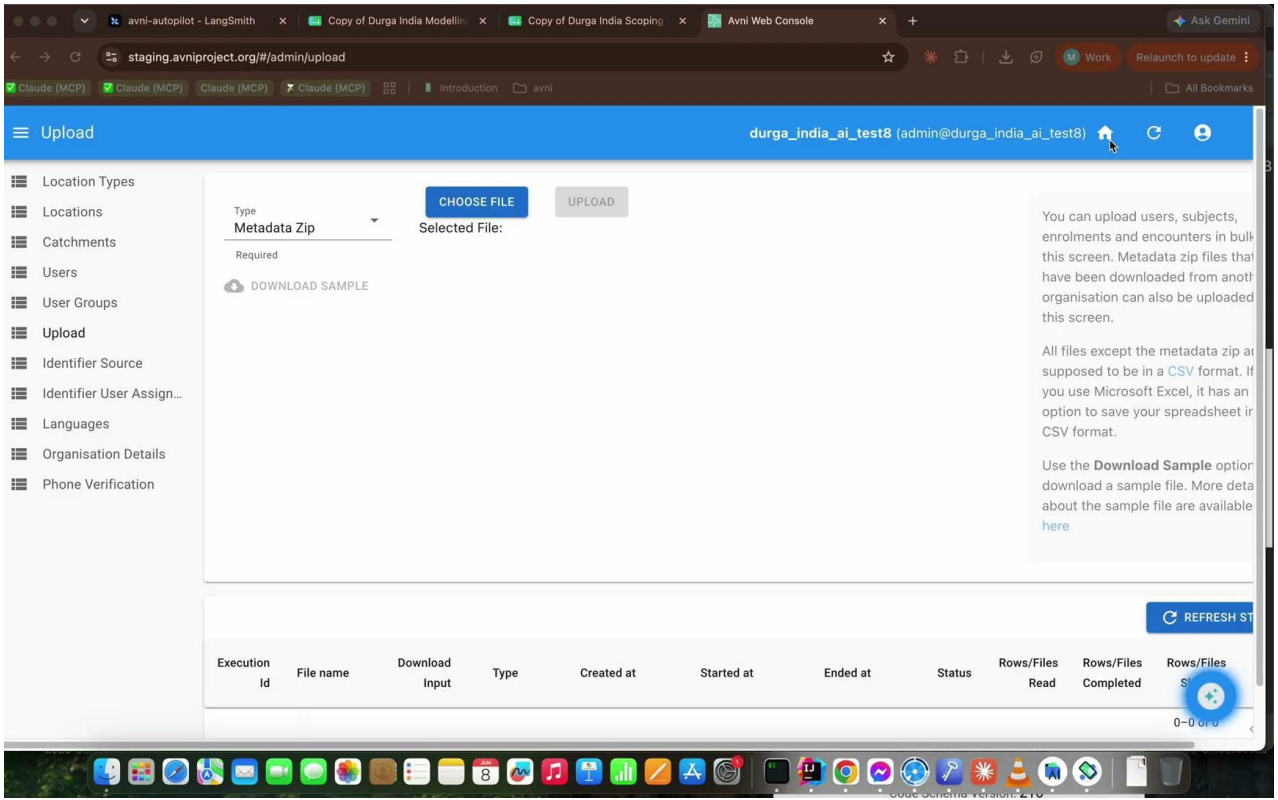


- Drag-and-drop visual nodes only — **difficult to design custom control flow** as needed
- Limited observability into anything that lives outside the canvas

Architecture diagram



Demonstration of the work done



Architectural learnings

What worked, picked deliberately.

LangGraph

FOR ORCHESTRATION

- **Observability** of every step in the run
- **Pause / resume** for human-in-the-loop review
- **Parallel + sequential** node execution side by side
- **Per-node retry** isolates failures
- **Chat-session state** persists across resumes

LLM vs deterministic

WHERE EACH ONE WINS

- Use the **LLM** when intelligent deduction is needed
- **Code it** when the path is clear and well-defined
- Never give the LLM **counting** tasks
- **Always validate** the LLM's output — syntax, grounding, schema

RAG

WHAT WE PICKED, WHAT MADE IT WORK

TECH CHOICES

- Small corpus (<10K items): **flat JSON + brute-force cosine** is enough
- **Voyage AI embeddings** perform strongly on code-heavy retrieval
- **Content-hash invalidation + lazy embed** — zero API cost when nothing changed

DATA PREP MADE IT WORK

- **Curate ruthlessly** — dedupe + paired prompts beat raw dumps
- **Annotate every helper** (signature, use_when, snippet)
- Garbage in, garbage out — output mirrors corpus quality

Roadmap

Six weeks of focused build, then phase 2.

1

JUN 12

Generate all rule types

Beyond visit schedule — eligibility, validation, summary, exit, form-element rules.

2

JUN 19

Fault tolerance

Catch and recover from common mistakes in the modelling doc — typos, missing references, inconsistent naming.

3

JUL 10

UI integration

Trigger bundle generation from Avni's web interface, surface progress and warnings inline.

4

JUL 17

Edit support

Update an already-generated bundle without losing field UUIDs or observation history.

5

PHASE 2

JUL 31

Unstructured inputs

Convert free-form requirements (notes, emails, conversations) into structured modelling documents.

Expected outcome

Before vs. after targets across the two transformation phases.

PHASE 1

Requirements → Specification documents

1.5–2 days → ~0.5 day

Today	**1.5–2 days of effort**
Target	**~0.5 day**

Scope: AI automates the document creation. Discovery calls with the implementation team still happen — the AI doesn't replace them.

PHASE 2

Specification documents → Avni app

30% less effort

Today	**100% of current effort**
Target	**~30% reduction**

Out of scope: field visits, online and offline reports (DB migration in progress), and project-management work — those continue at today's effort.

THANK YOU!